

Prueba Técnica Backend E-commerce

Tiempo Máximo de la Prueba: 90 Minutos

Stack Evaluable

Se evaluará el conocimiento profundo de las herramientas clave para la integración empresarial, la transformación de datos (MapStruct), la escritura de código limpio y la capacidad de diseñar un sistema resiliente ante fallos de red entre microservicios.

Contexto General de la Prueba Técnica: E-commerce Backend

El ejercicio consiste en desarrollar una solución basada en microservicios para el núcleo de una plataforma de comercio electrónico. El objetivo principal de la prueba es integrar información de dos fuentes distintas para exponer una API enriquecida que combine los detalles de un producto con su disponibilidad de inventario actual.

Se exige el uso de persistencia de datos real, orquestar la infraestructura de bases de datos utilizando herramientas de contenerización (Ej: Docker, podman).

Tecnología y Consideraciones

Requisitos Técnicos:

- Java 11+ (8/17/21 válido)
- Spring Boot
- Maven o Gradle
- Lombok
- Swagger/OpenAPI
- Docker
- Git

Consideraciones Generales:

- Los microservicios deben ser completamente independientes.
- El proyecto debe ser fácil de compilar y ejecutar.
- El evaluador no debe requerir instalaciones manuales complejas.
- Bases de Datos Permitidas: MongoDB.

Arquitectura Esperada - Diseño de la Solución

La solución se divide en dos dominios principales que se comunican de forma síncrona pero resiliente.

Arquitectura Esperada:

1. Product Service: Orquestador principal. Recibe el catálogo, transforma los datos externos con MapStruct y expone la API. Utiliza MongoDB.

2. Inventory Service: Servicio de soporte. Proporciona el stock actual por producto.
3. Comunicación: Se requiere el uso de clientes HTTP como OpenFeign, WebClient o RestClient.

Microservicio 1: Product Service

Responsabilidad: Administrar el catálogo de productos y realizar la transformación de datos inicial desde el archivo JSON externo a tu modelo interno.

Datos Mínimos y Persistencia: Debe almacenar id, nombre, descripción y precio en MongoDB (recomendado).

Endpoint Obligatorio: GET /api/v1/products/search?query={keyword}

Requerimiento de Búsqueda: Permitir buscar productos por coincidencias en nombre o descripción.

Integración Externa: Consultar al inventory-service para obtener el stock actual y responder con los productos enriquecidos.

Microservicio 2: Inventory Service

Responsabilidad: Administrar de forma aislada y eficiente el inventario de los productos disponibles.

Datos Mínimos y Persistencia: Debe gestionar productId y stock. Para agilizar la prueba, se sugiere el uso de H2 o almacenamiento estructurado en memoria.

Endpoint Obligatorio: GET /api/v1/inventory/{productId}

Consideración Técnica: Este servicio funge como una dependencia crítica para el catálogo. Su comportamiento debe ser predecible para facilitar las pruebas de resiliencia de red.

Manejo de Errores y Resiliencia

La solución debe manejar adecuadamente los fallos que ocurran en la red entre microservicios.

Escenario Esperado:

Si el inventory-service se cae, está lento o no responde, el product-service debe capturar esta falla y continuar respondiendo la búsqueda de forma controlada.

Ejemplo:

```
{  
  "id": "EXT-001",  
  "name": "iPhone 13 128GB",  
  "description": "Smartphone Apple..."
```

```
"price": 18999.99,  
"stock": null,  
"inventoryStatus": "UNAVAILABLE"  
}
```

Patrones Sugeridos: Timeout, Retry, Circuit Breaker, Fallback (usando Resilience4j u otro enfoque lógico).

Calidad, Testing y Entregables

Calidad de Código

Se evaluará estrictamente la separación de responsabilidades, el uso de DTOs, el manejo global de excepciones (@ControllerAdvice), el clean code y la mantenibilidad de la solución.

Testing

Se esperan pruebas unitarias en la lógica core de negocio. Las herramientas sugeridas son JUnit 5, Mockito o Spock para asegurar la integridad de la transformación y resiliencia.

Entregables Finales

Repositorio con código fuente, un README.md instrucciones claras para que, ejemplos curl/Postman y Swagger habilitado.

Plus: Docker Compose, CI/CD, Commits Semánticos.

Datos Dummy: Productos

Contexto del Archivo

El equipo funcional proporciona este archivo JSON (products-input.json) proveniente de un sistema legado.

Instrucción:

Debes procesar este archivo y mapear sus campos (usando MapStruct) hacia las entidades de tu product-service antes de almacenarlos en la BD.

products-input.json

```
[  
  {  
    "id": "EXT-001",  
    "title": "iPhone 13 128GB",  
    "description": "Smartphone Apple...",  
    "price": "18999.99",  
    "currency": "MXN",  
    "category": "smartphones",  
    "active": true  
  },  
]
```

```
{
  "id": "EXT-002",
  "title": "Samsung Galaxy S22",
  "description": "Gama alta...",
  "price": "17499.50",
  "currency": "MXN",
  "category": "smartphones",
  "active": true
},
{
  "id": "EXT-003",
  "title": "MacBook Air M1",
  "description": "Laptop ligera...",
  "price": "21999.00",
  "currency": "MXN",
  "category": "laptops",
  "active": true
},
{
  "id": "EXT-004",
  "title": "Dell XPS 13",
  "description": "Ultrabook premium...",
  "price": "24999.99",
  "currency": "MXN",
  "category": "laptops",
  "active": true
},
{
  "id": "EXT-005",
  "title": "Audífonos Sony WH-1000XM4",
  "description": "Cancelación de ruido",
  "price": "5999.00",
  "currency": "MXN",
  "category": "audio",
  "active": true
}
]
```

Datos Dummy: Inventario

Contexto del Archivo

Este archivo representa el estado actual del inventario (inventory-input.json).

Instrucción:

El inventory-service debe precargar estos datos en memoria o en H2 al arrancar, para poder atender las peticiones HTTP que realizará el catálogo de productos.

inventory-input.json

```
[
  {
    "productId": "EXT-001",
    "stock": 10
  },
  {
    "productId": "EXT-002",
    "stock": 0
  },
  {
    "productId": "EXT-003",
    "stock": 5
  },
  {
    "productId": "EXT-004",
    "stock": 2
  },
  {
    "productId": "EXT-005",
    "stock": 20
  }
]
```